NODEBASE

Node Reference & Operations Guide

How every trigger and action node works, how to configure it, and how to set it up end to end.

Workflow save + BullMQ cron scheduling | All 14 trigger/action nodes
Credential setup | Output variables | Troubleshooting

About this guide

This document is generated from the Nodebase source code. It describes the actual behavior of each node, not an idealized version. Where the UI text and the implementation differ, the difference is called out.

Use it as an operator manual: build a workflow, configure each node, connect the external service, save, and run.

Table of Contents

1. How Nodebase Runs a Workflow 4

1.1 The two layers 4

1.2 The execution pipeline 4

1.3 Node order and the context object 4

1.4 Realtime status 5

1.5 Saving a workflow 5

1.6 Failure behavior 5

2. Saving a Workflow and the Cron / BullMQ Flow 6

2.1 End-to-end flow 6

2.2 How a schedule becomes a cron pattern 6

2.3 The BullMQ queue configuration 7

2.4 The cron worker 7

2.5 What the Cron Trigger node does at execution time 7

2.6 Cleanup behavior 7

3. Trigger Nodes 8

3.1 Manual Trigger 8

3.2 Schedule / Cron Trigger 8

3.3 Google Form Trigger 9

3.4 Stripe Trigger 10

3.5 Telegram Trigger 11

4. Action Nodes 12

4.1 HTTP Request 12

4.2 OpenAI 13

4.3 Anthropic 13

4.4 Gemini 14

4.5 Discord 14

4.6 Slack 15

4.7 Send Telegram Message . . . 15

4.8 Send Telegram Message with Buttons . . . 16

4.9 Google Sheets . . . 17

5. Credentials and External Setup . . . 19

5.1 How to create a credential . . . 19

5.2 OpenAI API key . . . 19

5.3 Anthropic API key . . . 19

5.4 Gemini API key . . . 19

5.5 Telegram bot token and chat id . . . 19

5.6 Google Sheets service account . . . 20

6. Variables and Handlebars Templates . . . 21

6.1 Syntax . . . 21

6.2 Output variable reference . . . 21

7. Environment and Processes . . . 22

7.1 Required processes . . . 22

7.2 Environment variables that affect nodes . . . 22

8. Troubleshooting and Known Limitations . . . 23

8.1 Symptom to cause . . . 23

8.2 Known limitations and implementation notes . . . 23

8.3 Recommended build order . . . 23

Appendix A. Quick Reference . . . 24

Appendix B. Source Files . . . 24

1. How Nodebase Runs a Workflow

Nodebase is a visual workflow builder. A workflow is a graph of nodes stored in PostgreSQL. Trigger nodes start a run; action nodes do work. The editor is React Flow. The execution engine is Inngest. Scheduled workflows add a second layer: BullMQ repeatable jobs in Redis. This section explains the common machinery that every node relies on.

1.1 The two layers

Layer	Responsibility	Key files
Editor (client)	React Flow canvas, node dialogs, Save button, realtime status badges.	src/features/editor, src/config/node-components.ts
API (tRPC)	Persists workflows, nodes, edges, credentials, and manual execution.	src/features/workflows/server/routers.ts
Inngest (execution)	Runs executeWorkflow, creates Execution rows, executes nodes in order, publishes realtime status.	src/inngest/functions.ts
BullMQ + Redis (cron)	Stores repeatable schedules and fires an Inngest event when a schedule is due.	src/lib/queue/cron.ts, src/queue/cron-worker.ts

1.2 The execution pipeline

Every trigger, whether manual, webhook, or cron, ultimately sends the same Inngest event: workflows/execute.workflow. The event payload contains a workflowId and optionally initialData. The initialData becomes the starting context.

```
Trigger (manual / webhook / cron)
  -> inngest.send("workflows/execute.workflow", { workflowId, initialData? })
  -> executeWorkflow()
       step "create-execution"  -> INSERT Execution (status RUNNING)
       step "prepare-workflow"  -> load nodes + connections, topological sort
       step "find-user-id"      -> load workflow owner
       context = event.data.initialData || {}
       for each node in sorted order:
           executor({ data, nodeId, userId, context, step, publish })
           context = returned context
       step "update-execution"  -> status SUCCESS, completedAt, output = context
```

Important details from the implementation: production runs retry up to 3 times; development retries are 0. If the function fails, onFailure updates the Execution row to FAILED and stores the error and stack. Executors throw NonRetriableError for configuration mistakes, so those are not retried.

1.3 Node order and the context object

The engine does a topological sort of the graph using the stored connections. Disconnected nodes are given a self-edge so they still appear in the sorted list. That means every node in the workflow runs, even if it is not connected. A cycle throws Workflow contains a cycle and the run fails.

The context is a plain JavaScript object that accumulates results. A trigger puts its payload into the context; each action node returns a copy of the context plus its own output key. Later nodes can read earlier outputs with Handlebars syntax.

```
Initial context (Google Form trigger):
{
  googleForm: {
    respondentEmail: "a@b.com",
    responses: { "Name": "Ada", "Plan": "Pro" },
    formId: "...", formTitle: "...", responseId: "...", timestamp: "..."
  }
}

After an HTTP Request node with variableName "api":
{
  ...googleForm,
  api: {
    httpResponse: { status: 200, statusText: "OK", data: { id: 42 } }
  }
}

A later node can use: {{api.httpResponse.data.id}}
Or stringify an object: {{json googleForm.responses}}
```

1.4 Realtime status

Each node has an Inngest realtime channel. Executors publish a status object { nodeId, status } where status is loading, success, or error. The editor subscribes with a short-lived token and paints the node border. This is display only; it does not change execution.

1.5 Saving a workflow

The Save button reads the current React Flow nodes and edges and calls workflows.update. The update is a merge, not a delete-and-recreate:

- Existing nodes are upserted by id. Position and type are updated.
- Node data is only overwritten when the client sends a non-empty data object. This prevents an incomplete snapshot from wiping configuration.
- Nodes missing from the payload are deleted. Their connections cascade.
- All connections for the workflow are deleted and recreated from the edges array.
- The workflow updatedAt timestamp is refreshed.
- After the database transaction, the editor calls syncCronSchedules(workflowId).

Save is required before a trigger can use new configuration
Node dialogs update local React Flow state immediately. They do not write to the database. Inngest and server actions read the database. If you configure a node and do not press the workflow Save button, the run will use the old data. This matters most for the Telegram trigger's Register Webhook button, which reads the saved node.

1.6 Failure behavior

Situation	What happens
Missing required node field	Executor publishes error and throws NonRetriableError. The run fails.
HTTP endpoint returns non-2xx	ky throws by default, so the HTTP Request node fails the run.
External API rejects credentials	Executor throws; Inngest retries in production, then onFailure marks FAILED.
Cycle in graph	topologicalSort throws Workflow contains a cycle before any node runs.
Execution record cannot be created	create-execution throws NonRetriableError.

2. Saving a Workflow and the Cron / BullMQ Flow

This is the full path from pressing Save to a scheduled workflow actually running. The Cron Trigger node is unusual: it does almost nothing during execution. The real scheduling happens when the workflow is saved, through BullMQ and Redis.

2.1 End-to-end flow

```
[Editor] Save button
  -> trpc workflows.update
       -> Postgres: upsert nodes, replace connections
  -> syncCronSchedules(workflowId)                 (server action)
       -> auth session check
       -> load workflow + nodes
       -> for each CRON_TRIGGER node:
            registerCronSchedules(workflowId, nodeId, schedules)
              -> getCronQueue()                    (BullMQ Queue "cron")
              -> for each schedule:
                   scheduleToCron(schedule)        (preset / cron / interval)
                   cronQueue.upsertJobScheduler(
                     "cron:<nodeId>:<scheduleId>",
                     { pattern, tz, startDate?, endDate? },
                     { name: "cron", data: { workflowId, nodeId, scheduleId, timezone } }
                   )
       -> findStaleCronNodeIds()
            -> queue.getJobSchedulers()
            -> remove schedulers whose nodeId no longer exists in this workflow
  -> Redis now holds repeatable job schedulers

[At the scheduled time]
  -> BullMQ creates a job on the "cron" queue
  -> cron-worker.ts (separate Node process) picks it up
  -> inngest.send("workflows/execute.workflow", {
       workflowId,
       initialData: { cron: { nodeId, scheduleId, timezone, timestamp } }
     })
  -> executeWorkflow() runs normally
```

2.2 How a schedule becomes a cron pattern

The dialog supports three kinds. scheduleToCron converts each to a standard five-field cron expression. The timezone is passed separately to BullMQ, not embedded in the expression.

Dialog kind	Input	Generated pattern
Preset everyMinute	No extra input	* * * * *
Preset hourly	No extra input	0 * * * *
Preset daily	hour, minute	<minute> <hour> * * *
Preset weekly	hour, minute, dayOfWeek[]	<minute> <hour> * * <days>
Preset monthly	hour, minute, dayOfMonth	<minute> <hour> <dayOfMonth> * *
Custom cron	cronExpression	Used exactly as entered
Interval minutes	every = N	*/N * * * *
Interval hours	every = N	0 */N * * *

Custom expressions are validated in the dialog with cron-parser. The dialog also supports optional startDate and endDate. Both are passed to the BullMQ repeat options. Multiple schedules can exist on one Cron Trigger node; each has its own random UUID and therefore its own scheduler key.

2.3 The BullMQ queue configuration

```
Queue name: "cron"
Connection: ioredis from UPSTASH_REDIS_URL
  maxRetriesPerRequest: null
  enableReadyCheck: false
Default job options:
  attempts: 3
  backoff: exponential, 5000 ms
  removeOnComplete: true
  removeOnFail: 100
Scheduler key: cron:<nodeId>:<scheduleId>
Worker concurrency: 5
```

If UPSTASH_REDIS_URL is not set, getCronQueue returns null. Saving a workflow still succeeds, but no schedules are registered. The cron worker exits immediately with a warning when the URL is missing.

2.4 The cron worker

The worker is a separate process. It is not started by Next.js. In development, npm run dev:all starts it through mprocs. In production it must be deployed as its own always-on service. The worker sends the Inngest event; it does not execute nodes itself.

```
npm run cron-worker
  -> tsx src/queue/cron-worker.ts
  -> new Worker("cron", async (job) => {
       const { workflowId, nodeId, scheduleId, timezone } = job.data;
       await inngest.send({
         name: "workflows/execute.workflow",
         data: {
           workflowId,
           initialData: {
             cron: { nodeId, scheduleId, timezone, timestamp: new Date().toISOString() }
           }
         }
       });
     }, { connection, concurrency: 5 })
```

2.5 What the Cron Trigger node does at execution time

When the scheduled event reaches Inngest, the Cron Trigger node is just another node in the sorted list. Its executor publishes loading, runs a no-op step that returns the existing context, and publishes success. The cron metadata is already in the context from initialData. Downstream nodes can read it with {{cron.nodeId}}, {{cron.scheduleId}}, {{cron.timezone}}, and {{cron.timestamp}}.

2.6 Cleanup behavior

- Deleting a workflow calls removeWorkflowCronSchedulers for all of its Cron Trigger node ids. Every scheduler whose key starts with cron:<nodeId>: is removed.
- Saving a workflow removes schedulers whose node id is no longer present. This handles deleting a Cron Trigger node.
- Re-saving the same schedule calls upsertJobScheduler with the same key, so it replaces the existing schedule instead of duplicating it.

Known limitation: removing one schedule from an existing cron node
Cleanup is node-level, not schedule-level. If a Cron Trigger node stays in the workflow but you delete one of its schedules, the old scheduler key remains in Redis because the node id still exists. Workaround: delete the whole Cron Trigger node and save (this removes all of its schedulers), then add a new one. Or remove the stale key manually with a Redis/BullMQ maintenance script.

3. Trigger Nodes

Trigger nodes start a workflow. The editor prevents more than one Manual Trigger and more than one Cron Trigger per workflow. Webhook triggers are started by external services. All trigger executors are pass-through: they publish status and return the existing context. Their real work happens in the API route or scheduler before Inngest is called.

3.1 Manual Trigger

Property	Value
Node type	MANUAL_TRIGGER
Starts when	User clicks Execute workflow in the editor
Configuration	None
Initial context	{} (empty object)

How it works

1. Add the Manual Trigger node. There is no dialog configuration.
2. The editor only shows the Execute workflow button when at least one MANUAL_TRIGGER node exists in the canvas.
3. Clicking the button calls workflows.execute, which sends workflows/execute.workflow with only the workflowId.
4. The engine creates an Execution, starts with an empty context, and runs the Manual Trigger executor.
5. The executor publishes loading, runs a no-op step, publishes success, and passes the empty context to downstream nodes.

Setup

- Add the node from the node selector.
- Connect it to the first action node.
- Press Save, then press Execute workflow.

Note
If the Execute workflow button is missing, the workflow has no Manual Trigger node. Add one, or use a webhook/cron trigger instead.

3.2 Schedule / Cron Trigger

Property	Value
Node type	CRON_TRIGGER
Starts when	A BullMQ repeatable job fires
Configuration	One or more schedules + timezone + optional start/end date
Initial context	{ cron: { nodeId, scheduleId, timezone, timestamp } }

Configuration fields

Field	Required	Description
Schedule kind	Yes	Preset, Custom Cron, or Interval. Selected with the three buttons at the top of the dialog.
Preset type	Preset only	Every minute, Hourly, Daily, Weekly, Monthly. Daily/Weekly/Monthly need an hour and minute. Weekly also needs at least one day. Monthly needs a day of month (1-31).
Cron expression	Custom only	Five fields: minute hour day-of-month month day-of-week. Example: 0 9 * * 1-5 for weekdays at 09:00. Validated with cron-parser.
Interval	Interval only	Every N minutes or every N hours. N is 1-999.
Timezone	Yes	IANA timezone, for example UTC, Asia/Kolkata, Europe/London.
Start date	No	The schedule does not fire before this date.
End date	No	The schedule stops after this date. Must be after start date.
Add another schedule	No	Adds a second independent schedule to the same node. Each has its own UUID.

Setup and full flow

1. Add Schedule / Cron from the node selector.
2. Open the node. Choose Preset, Custom Cron, or Interval.
3. Fill the fields and choose a timezone. Optionally set start/end dates.
4. Press Save in the dialog. This only updates local editor state.
5. Press the workflow Save button. This writes the node to Postgres and calls syncCronSchedules.
6. syncCronSchedules registers each schedule in the BullMQ cron queue using upsertJobScheduler.
7. Run the cron worker (npm run cron-worker, or npm run dev:all). It must be running for jobs to fire.
8. At the scheduled time, the worker sends the Inngest event with the cron initialData.
9. executeWorkflow runs the Cron Trigger executor (pass-through) and then the downstream nodes.

Requirements
UPSTASH_REDIS_URL must be an ioredis-compatible Redis URL. The cron worker must be running as a separate process. If either is missing, the workflow will never fire on schedule even though the node looks configured.

3.3 Google Form Trigger

Property	Value
Node type	GOOGLE_FORM_TRIGGER
Starts when	A Google Form Apps Script POSTs a submission
Webhook	/api/webhooks/google-form?workflowId=<id>
Initial context	{ googleForm: { formId, formTitle, responseId, timestamp, respondentEmail, responses, raw } }

Configuration fields

Field	Required	Description
Webhook URL	Generated	Shown read-only in the dialog. Copy it into the Apps Script.
Google Apps Script	Yes	Copy the generated script and paste it into the form's Apps Script project.

Setup

1. Add the Google Form trigger node and open it.
2. Click Copy Google Apps Script. The script already contains the webhook URL.
3. Open the Google Form, click the three-dot menu, then Apps Script.
4. Paste the script, save, and open Triggers.
5. Add Trigger: choose From form, On form submit, then Save. Authorize the script.
6. Submit a test response. The script POSTs JSON to the webhook.
7. The route checks the workflowId query parameter, applies the rate limit, builds googleForm initialData, and calls sendWorkflowExecution.

8. The Inngest event runs the workflow with the googleForm context.

Available variables

Variable	Meaning
{{googleForm.respondentEmail}}	Respondent email, if the form collects it
{{googleForm.responses['Question Name']}}	One answer, keyed by question title
{{json googleForm.responses}}	All answers as pretty-printed JSON
{{googleForm.formTitle}}	Form title
{{googleForm.responseId}}	Unique response id; also used for idempotency
{{googleForm.timestamp}}	Submission timestamp

Deployment note
Google Apps Script must be able to reach the webhook URL. In local development that means a public tunnel such as ngrok, and NEXT_PUBLIC_APP_URL must point at it. The route does not verify a Google signature; anyone who knows the URL can POST. Rate limiting and responseId idempotency reduce abuse but do not authenticate.

3.4 Stripe Trigger

Property	Value
Node type	STRIPE_TRIGGER
Starts when	Stripe POSTs an event to the webhook endpoint
Webhook	/api/webhooks/stripe?workflowId=<id>
Initial context	{ stripe: { eventId, eventType, timestamp, livemode, raw } }

Setup

1. Add the Stripe trigger node and open it.
2. Copy the generated webhook URL.
3. In the Stripe Dashboard go to Developers > Webhooks > Add endpoint.
4. Paste the URL and select the events you want, for example payment_intent.succeeded.
5. Save the endpoint. Stripe starts sending events.
6. The route reads workflowId, applies the rate limit, builds stripe initialData, and calls sendWorkflowExecution.
7. The workflow runs with the stripe context.

Actual available variables

The dialog lists {{stripe.amount}}, {{stripe.currency}}, and {{stripe.customerId}}, but the route does not put those fields at the top level. It puts the Stripe event object under stripe.raw. Use the paths below.

Variable	Meaning
{{stripe.eventId}}	Stripe event id
{{stripe.eventType}}	Event type, e.g. payment_intent.succeeded
{{stripe.timestamp}}	Event created time (Unix seconds)
{{stripe.livemode}}	Whether the event is from live mode
{{stripe.raw.amount}}	Amount from the event object, when present
{{stripe.raw.currency}}	Currency from the event object, when present
{{stripe.raw.customer}}	Customer id from the event object, when present
{{json stripe.raw}}	Full event object as JSON

Implementation note: no signature verification
The Stripe route does not verify the Stripe-Signature header and does not use a signing secret. The dialog mentions copying the signing secret, but the current code does not read it. Treat the webhook URL as the only access control and keep rate limits enabled. Add signature verification before exposing this publicly if you need authenticated Stripe events.

3.5 Telegram Trigger

Property	Value
Node type	TELEGRAM_TRIGGER
Starts when	A new Telegram message arrives for the registered bot
Webhook	/api/telegram/webhook/<workflowId>
Configuration	Telegram bot credential + Register Webhook
Initial context	{ telegram: { chat, text, from, date, messageId, raw } }

Configuration fields

Field	Required	Description
Telegram Bot Credential	Yes	A credential of type TELEGRAM containing the bot token from BotFather.
Register Webhook button	Yes	Calls Telegram setWebhook for this workflow. Must be clicked after the workflow has been saved, because the server action reads the saved node.

Setup

1. Create a bot with @BotFather: send /newbot, choose a name and a username ending in bot, and copy the token (format 123456789:AA...).
2. In Nodebase go to Credentials > New credential. Choose type Telegram and paste the token. Save it.
3. Add the Telegram trigger node and select the credential.
4. Press the workflow Save button.
5. Reopen the trigger and click Register Webhook.
6. The server action computes the webhook URL from NEXT_PUBLIC_APP_URL, creates a secret token with HMAC-SHA256(workflowId, key=bot token), and calls Telegram setWebhook with drop_pending_updates.
7. It calls getMe to show the bot username and warns if the same credential is used by another workflow.
8. Send a message to the bot. Telegram POSTs the update to the route.
9. The route verifies the x-telegram-bot-api-secret-token header, ignores callback queries after answering them, and sends the workflow event for normal messages.

Available variables

Variable	Meaning
{{telegram.chat.id}}	Chat id; use this to reply to the same chat
{{telegram.text}}	Message text
{{telegram.from.id}}	Sender user id
{{telegram.from.username}}	Sender username
{{telegram.from.first_name}}	Sender first name
{{telegram.from.last_name}}	Sender last name
{{telegram.chat.type}}	private, group, supergroup, or channel
{{telegram.chat.title}}	Group/channel title, when present
{{telegram.messageId}}	Telegram message id
{{telegram.date}}	Message date (Unix seconds)
{{json telegram}}	Full normalized update as JSON
{{json telegram.raw}}	Raw Telegram message object

Telegram trigger limitations
Only body.message starts a workflow. Edits, channel posts, and other update types are ignored. Button taps (callback_query) are acknowledged with answerCallbackQuery and then return without starting a workflow. A bot can have only one webhook: registering another workflow with the same bot overwrites the previous webhook. Re-register after changing the credential or the public URL.

4. Action Nodes

Action nodes run after a trigger. Each one receives the current context, validates its configuration, does its work, and returns the context plus a new output key. The output key is the node's Variable Name field. If a required field is missing, the node publishes error and throws NonRetriableError.

4.1 HTTP Request

Property	Value
Node type	HTTP_REQUEST
Purpose	Call any HTTP API
Output	{{<variableName>.httpResponse.{status,statusText,data}}}

Configuration fields

Field	Required	Description
Variable Name	Yes	Name used to store the response. Must start with a letter, underscore, or dollar sign and contain only letters, numbers, underscores, or dollar signs. Example: myApiCall.
Method	Yes	GET, POST, PUT, PATCH, or DELETE. Default is GET.
Endpoint URL	Yes	Static URL or a Handlebars template. Example: https://api.example.com/users/{{googleForm.responses['UserId']}}
Request Body	POST/PUT/PATCH only	JSON text with Handlebars variables. Shown only for methods that can have a body. Defaults to {} if left empty.

How it works

1. Publishes loading.
2. Validates endpoint, variableName, and method. Missing values throw NonRetriableError.
3. Compiles the endpoint with Handlebars using the current context.
4. For POST, PUT, and PATCH, compiles the body, validates it with JSON.parse, and sends it as the raw JSON string with Content-Type: application/json.
5. Calls the endpoint with ky. ky throws on non-2xx responses, so 4xx and 5xx fail the node.
6. Reads the response as JSON when content-type includes application/json; otherwise reads text.
7. Stores { httpResponse: { status, statusText, data } } under the variable name.
8. Publishes success, or error and rethrows.

Example

```
Variable Name: githubUser
Method: GET
Endpoint: https://api.github.com/users/octocat

Later node: {{githubUser.httpResponse.data.name}}
Status code: {{githubUser.httpResponse.status}}
Full payload: {{json githubUser.httpResponse.data}}

POST example body:
{
  "name": "{{googleForm.responses['Name']}}",
  "email": "{{googleForm.respondentEmail}}"
}
```

Current limitation: no custom headers or auth field
The node sends only Content-Type for JSON bodies. It has no headers table and no separate authentication field. If an API needs a bearer token or API key header, you must extend the executor or use an endpoint that accepts credentials in the URL/body. Also note that non-2xx responses fail the run by default.

4.2 OpenAI

Property	Value
Node type	OPENAI
Purpose	Generate text with OpenAI
Model	gpt-4 (hard-coded)
Output	{{<variableName>.text}}

Configuration fields

Field	Required	Description
Variable Name	Yes	Output key. Example: myOpenAi.
OpenAI Credential	Yes	A credential of type OPENAI.
System Prompt	No	Behavior instructions. Defaults to: You are a helpful assistant.
User Prompt	Yes	The prompt. Supports Handlebars variables.

Setup

1. Create an API key at platform.openai.com: API keys > Create new secret key. Copy it; it is shown once.
2. In Nodebase, create a credential of type OpenAI and paste the key.
3. Add the OpenAI node, select the credential, and write the prompts.
4. Use {{json someVariable}} when you want to insert a whole object into the prompt.

How it works

1. Publishes loading.
2. Validates variableName, credentialId, and userPrompt.
3. Compiles the system and user prompts with the current context.
4. Loads the credential by id and userId, then decrypts it with ENCRYPTION_KEY.
5. Calls step.ai.wrap with generateText, model gpt-4, telemetry enabled.
6. Extracts the first text block and stores it as { text } under the variable name.

Model availability
The model string is hard-coded as gpt-4. If your OpenAI account does not have access to that model, the node fails. Change the executor if you need a different model.

4.3 Anthropic

Property	Value
Node type	ANTHROPIC
Purpose	Generate text with Anthropic Claude
Model	claude-sonnet-4-0 (hard-coded)
Output	{{<variableName>.text}}

Configuration fields

Field	Required	Description
Variable Name	Yes	Output key. Example: myAnthropic.
Anthropic Credential	Yes	A credential of type ANTHROPIC.
System Prompt	No	Defaults to: You are a helpful assistant.
User Prompt	Yes	Supports Handlebars variables.

Setup

1. Open console.anthropic.com and create an API key under API Keys.

2. Create a credential of type Anthropic in Nodebase and paste the key.
3. Add the node, select the credential, and configure the prompts.

Execution is the same as OpenAI: validate, compile prompts, load and decrypt the credential, call step.ai.wrap with generateText, store the text. The only differences are the SDK, the model, and the credential type.

4.4 Gemini

Property	Value
Node type	GEMINI
Purpose	Generate text with Google Gemini
Model	gemini-flash-latest (hard-coded)
Output	{{<variableName>.text}}

Configuration fields

Field	Required	Description
Variable Name	Yes	Output key. Example: myGemini.
Gemini Credential	Yes	A credential of type GEMINI.
System Prompt	No	Defaults to: You are a helpful assistant.
User Prompt	Yes	Supports Handlebars variables.

Setup

1. Open aistudio.google.com and create an API key.
2. Create a credential of type Gemini in Nodebase and paste the key.
3. Add the node, select the credential, and configure the prompts.

Execution mirrors the other AI nodes: compile prompts, load and decrypt the credential, call generateText with gemini-flash-latest, store { text } under the variable name.

4.5 Discord

Property	Value
Node type	DISCORD
Purpose	Send a message through a Discord webhook
Output	{{<variableName>.messageContent}}

Configuration fields

Field	Required	Description
Variable Name	Yes	Output key. Example: myDiscord.
Webhook URL	Yes	Discord channel webhook URL, starting with https://discord.com/api/webhooks/.
Message Content	Yes	Message text, maximum 2000 characters. Supports Handlebars.
Bot Username	No	Overrides the webhook's default display name.

Setup

1. In Discord, open the target server and channel.
2. Edit Channel > Integrations > Webhooks > New Webhook (or Server Settings > Integrations > Webhooks).
3. Give it a name, choose the channel, and copy the webhook URL.
4. Paste the URL into the node. Optionally set a username.
5. Send a test message by running the workflow.

How it works

1. Validates that content exists.
2. Compiles content and username with Handlebars, then decodes HTML entities.
3. Requires the webhook URL.
4. POSTs JSON { content, username } to the webhook.
5. Truncates content to 2000 characters.
6. Requires HTTP 204. Any other status is a NonRetriableError.
7. Stores { messageContent } under the variable name.

Dialog hint vs actual output
The dialog suggests {{myDiscord.text}}, but the executor stores the result under messageContent. Use {{myDiscord.messageContent}}.

4.6 Slack

Property	Value
Node type	SLACK
Purpose	Send a message through a Slack Incoming Webhook
Output	{{<variableName>.messageContent}}

Configuration fields

Field	Required	Description
Variable Name	Yes	Output key. Example: mySlack.
Webhook URL	Yes	Slack Incoming Webhook URL, starting with https://hooks.slack.com/services/.
Message Content	Yes	Message text. Supports Handlebars.

Setup

1. Go to api.slack.com/apps and create an app (From scratch).
2. Open Features > Incoming Webhooks and activate it.
3. Click Add New Webhook to Workspace, choose a channel, and allow access.
4. Copy the generated webhook URL.
5. Paste it into the node and write the message content.

How it works

1. Validates that content exists.
2. Compiles content with Handlebars and decodes HTML entities.
3. Requires the webhook URL.
4. POSTs JSON { text: content } to the webhook.
5. Requires HTTP 200 and a response body exactly equal to ok. Anything else is a NonRetriableError.
6. Stores { messageContent } under the variable name, truncated to 2000 characters.

Dialog hint vs actual output
The dialog suggests {{mySlack.text}}, but the executor stores messageContent. Use {{mySlack.messageContent}}. Slack's response must be exactly ok, which is normal for Incoming Webhooks.

4.7 Send Telegram Message

Property	Value
Node type	TELEGRAM_SEND_MESSAGE
Purpose	Send a text message with a Telegram bot
Output	{{<variableName>.telegramMessage}} (Telegram Message object)

Configuration fields

Field	Required	Description
Variable Name	Yes	Output key. Example: myTelegramMessage.
Telegram Bot Credential	Yes	A credential of type TELEGRAM.
Chat ID	Yes	Numeric chat id, @channelusername, or a template such as {{telegram.chat.id}}.
Message Text	Yes	Message body. Supports Handlebars.
Parse Mode	Yes	None, Markdown, or HTML. Default None. Markdown is converted to Telegram-compatible HTML automatically.

How it works

1. Validates variableName, credentialId, chatId, and text.
2. Compiles chatId and text with the context.
3. If Parse Mode is Markdown, converts Markdown to Telegram HTML and sends parse_mode=HTML. If HTML, sends parse_mode=HTML. If None, sends no parse mode.
4. Loads the bot credential and decrypts the token.
5. Calls the Bot API method sendMessage with chat_id, text, and optional parse_mode.
6. Numeric chat ids are converted from string to number. Usernames stay strings.
7. If Telegram returns 400 with can't parse entities and a parse mode was used, retries once as plain text so the message is still delivered.
8. Stores the Telegram Message object under telegramMessage.

Finding the chat id

The chat id is not the bot token and it is not the bot username. It identifies the conversation. Use one of these methods.

1. Private chat: open the bot in Telegram, press Start, and send any message.
2. In a browser, open https://api.telegram.org/bot<TOKEN>/getUpdates.
3. Find the message.chat.id value in the JSON. That is your chat id.
4. Group chat: add the bot to the group, send a message in the group, then call getUpdates. Group ids are negative, often starting with -100.
5. Channel: add the bot as an administrator, post in the channel, then call getUpdates. You can also use @channelusername if the bot can post there.
6. Alternative: use the Telegram Trigger node once and read {{telegram.chat.id}} from an incoming message.
7. If getUpdates returns 409 or an empty result, a webhook may already be set. Call https://api.telegram.org/bot<TOKEN>/deleteWebhook, then try again.

Bot must be allowed to talk to the chat
For private chats the user must press Start. For groups the bot must be a member. For channels the bot must be an administrator with posting rights. Otherwise Telegram returns chat not found or not enough rights.

4.8 Send Telegram Message with Buttons

Property	Value
Node type	TELEGRAM_SEND_BUTTONS
Purpose	Send a message with an inline keyboard
Output	{{<variableName>.telegramMessage}} (Telegram Message object)

Configuration fields

Field	Required	Description
Variable Name	Yes	Output key. Example: myTelegramButtons.
Telegram Bot Credential	Yes	A credential of type TELEGRAM.
Chat ID	Yes	Same rules as Send Telegram Message.
Message Text	Yes	Message body. Supports Handlebars.
Parse Mode	Yes	None, Markdown, or HTML.
Buttons	Yes	At least one button. Each has a Label and a Callback value. The callback value is limited to 64 characters.

How it works

1. Validates variableName, credentialId, chatId, text, and at least one button.
2. Compiles chatId, text, every button label, and every callback value with the context.
3. Applies the same Markdown/HTML handling as Send Telegram Message.
4. Builds reply_markup.inline_keyboard as one row containing all buttons.
5. Calls sendMessage with the inline keyboard.
6. On a parse-entities error, retries as plain text but keeps the buttons.
7. Stores the Telegram Message object under telegramMessage.

Important: button taps do not continue the workflow
The Telegram webhook route handles callback_query by calling answerCallbackQuery and returning. It does not start a workflow and does not branch on the callback value. The buttons are delivered to Telegram, but tapping them currently only acknowledges the tap. Do not build an approval flow that depends on the callback unless you extend the webhook route.

4.9 Google Sheets

Property	Value
Node type	GOOGLE_SHEETS
Purpose	Read, append, or update rows in a Google Sheet
Output	{{<variableName>.googleSheets}}

Configuration fields

Field	Required	Description
Variable Name	Yes	Output key. Example: mySheetsData.
Google Sheets Credential	Yes	A credential of type GOOGLE_SHEETS containing a service account JSON.
Operation	Yes	Append Row, Read Rows, or Update Cell.
Spreadsheet ID	Yes	The id from the sheet URL, or a Handlebars template.
Range	Yes	Sheet name and range. Examples: Sheet1!A1:C or Sheet1!A1.
Row Values	Append/Update only	One value per cell. Each value supports Handlebars. Not shown for Read.

Setup

1. Open console.cloud.google.com and create or select a project.
2. Enable the Google Sheets API for the project.
3. Go to IAM & Admin > Service Accounts > Create Service Account.
4. Give it a name and grant Editor (or Viewer for read-only).
5. Open the service account, go to Keys, Add Key, Create new key, choose JSON, and download the file.
6. In Nodebase, create a credential of type Google Sheets and paste the entire JSON file contents.
7. Open the target spreadsheet, click Share, and share it with the service account email (the client_email in the JSON) as Editor or Viewer.

8. Copy the Spreadsheet ID from the URL: https://docs.google.com/spreadsheets/d/<ID>/edit.
9. Add the node, select the credential, choose the operation, and enter the id, range, and values.

How it works

1. Validates variableName, credentialId, operation, spreadsheetId, and range.
2. For append and update, requires at least one row value.
3. Compiles spreadsheetId and range with the context.
4. Loads and decrypts the service account JSON.
5. Creates a GoogleSheetsClient using a JWT with the spreadsheets scope.
6. Read: calls spreadsheets.values.get and returns { spreadsheetId, range, values }.
7. Append: calls spreadsheets.values.append with USER_ENTERED and returns { spreadsheetId, range, updatedRows, updatedCells }.
8. Update: calls spreadsheets.values.update with USER_ENTERED and returns the same shape as append.
9. Each row value is compiled with Handlebars and trimmed before being written.
10. Stores the result under googleSheets.

Common Google Sheets failures
403 usually means the API is not enabled, the sheet is not shared with the service account, or the service account lacks Editor permission for writes. Invalid JSON or a missing client_email/private_key produces a credential error. The service account private key is encrypted at rest, so ENCRYPTION_KEY must stay stable.

5. Credentials and External Setup

Credentials are stored in the Credential table, encrypted with Cryptr using ENCRYPTION_KEY. A node stores only the credential id in its data. Executors load the credential by id and userId and decrypt it at run time. The decrypted value is never sent to the browser. Creating a credential uses premiumProcedure, so an active subscription is required; reading and deleting are available to the owner.

Credential type	Value to paste	Used by
OPENAI	OpenAI API key	OpenAI node
ANTHROPIC	Anthropic API key	Anthropic node
GEMINI	Google AI Studio API key	Gemini node
TELEGRAM	Bot token from BotFather	Telegram trigger, Send Message, Send Buttons
GOOGLE_SHEETS	Service account JSON	Google Sheets node

5.1 How to create a credential

1. Open Credentials in the sidebar and click New credential.
2. Enter a Name. This is only a label.
3. Choose the Type. The type must match the node that will use it.
4. Paste the Value. For Google Sheets this is the whole JSON file, not just the private key.
5. Click Create. The value is encrypted before it is stored.
6. Open the node that needs it and select the credential from the dropdown.

5.2 OpenAI API key

1. Go to platform.openai.com and sign in.
2. Open API keys and click Create new secret key.
3. Copy the key immediately; it is shown only once.
4. Paste it into a credential of type OpenAI.

5.3 Anthropic API key

1. Go to console.anthropic.com and sign in.
2. Open API Keys and create a new key.
3. Copy the key and paste it into a credential of type Anthropic.

5.4 Gemini API key

1. Go to aistudio.google.com.
2. Open Get API key and create a key for a Google Cloud project.
3. Copy the key and paste it into a credential of type Gemini.

5.5 Telegram bot token and chat id

Create the bot token

1. Open Telegram and search for @BotFather. Make sure it is the verified account.
2. Send /newbot.
3. Choose a display name, then a username that ends in bot.
4. BotFather replies with a token such as 123456789:AAH... . Copy it.
5. Paste it into a credential of type Telegram.

Find the chat id

1. Send a message to the bot from the target private chat, or add the bot to the target group/channel and post there.
2. Open https://api.telegram.org/bot<TOKEN>/getUpdates in a browser.

3. Find the chat object inside the message. The id field is the chat id.
4. Private chat ids are positive. Group and supergroup ids are negative.
5. For a channel, use @channelusername or the numeric id from getUpdates.
6. If getUpdates returns 409 Conflict, a webhook is already registered. Call https://api.telegram.org/bot<TOKEN>/deleteWebhook first.
7. You can also add a Telegram Trigger, send a message, and use {{telegram.chat.id}} in a downstream Send Telegram Message node.

Group privacy mode
By default a bot in a group only receives commands and replies addressed to it. If the trigger should see every message, open @BotFather, use /setprivacy, select the bot, and disable privacy mode. Then remove and re-add the bot to the group, or make it an administrator.

5.6 Google Sheets service account

1. Create or select a Google Cloud project.
2. Enable the Google Sheets API.
3. Create a service account under IAM & Admin > Service Accounts.
4. Grant Editor for read/write, or Viewer for read-only.
5. Create a JSON key and download it.
6. Paste the entire JSON into a credential of type Google Sheets.
7. Share the spreadsheet with the service account email as Editor or Viewer.
8. Copy the spreadsheet id from the URL.

The client only needs client_email and private_key from the JSON, but paste the whole file so it stays valid. The scope used is https://www.googleapis.com/auth/spreadsheets.

6. Variables and Handlebars Templates

Most text fields support Handlebars templates. The template is compiled at execution time against the current context. The context grows as nodes run, so a node can only reference variables produced by earlier nodes.

6.1 Syntax

Syntax	Result
{{variable}}	Insert a simple value
{{variable.path.to.field}}	Read a nested field
{{responses['Question Name']}}	Read a key with spaces or punctuation
{{json variable}}	Pretty-print the whole object as JSON
{{variable.httpResponse.status}}	HTTP status code
{{variable.text}}	AI output text
{{variable.telegramMessage.message_id}}	Telegram message id
{{variable.googleSheets.values}}	Google Sheets read result

The json helper is registered by the executors that need it. It returns a SafeString, so the JSON is not HTML-escaped. For plain values, Handlebars applies its normal escaping. If a value contains characters that break a URL or JSON payload, pass the whole object with {{json ...}} and parse it downstream, or adjust the executor.

6.2 Output variable reference

Node	Output path	Shape
HTTP Request	{{var.httpResponse}}	status, statusText, data
OpenAI	{{var.text}}	string
Anthropic	{{var.text}}	string
Gemini	{{var.text}}	string
Discord	{{var.messageContent}}	string, max 2000 chars
Slack	{{var.messageContent}}	string, max 2000 chars
Telegram Send Message	{{var.telegramMessage}}	Telegram Message object
Telegram Send Buttons	{{var.telegramMessage}}	Telegram Message object
Google Sheets	{{var.googleSheets}}	operation-specific result object
Cron Trigger	{{cron}}	nodeId, scheduleId, timezone, timestamp
Google Form Trigger	{{googleForm}}	form and response data
Stripe Trigger	{{stripe}}	eventId, eventType, timestamp, livemode, raw
Telegram Trigger	{{telegram}}	chat, text, from, date, messageId, raw

7. Environment and Processes

Some nodes depend on processes and environment variables outside Next.js. This section lists what must be running and why.

7.1 Required processes

Process	Command	Why it matters
Next.js	npm run dev	Serves the editor, tRPC API, webhooks, and the Inngest endpoint.
Inngest	npm run inngest:dev	Runs executeWorkflow in development. Functions are served at /api/inngest.
Cron worker	npm run cron-worker	Picks up BullMQ cron jobs and sends the Inngest event.
Email worker	npm run worker	Separate OTP email worker; not used by workflow nodes.
Tunnel	npm run ngrok:dev	Gives Google Forms, Stripe, and Telegram a public HTTPS URL in development.
All of the above	npm run dev:all	mprocs starts ngrok, Inngest, email worker, cron worker, and Next.js.

7.2 Environment variables that affect nodes

Variable	Used for
DATABASE_URL	Postgres, workflow and execution storage
ENCRYPTION_KEY	Encrypting and decrypting credentials
INNGEST_EVENT_KEY	Sending workflow events
INNGEST_SIGNING_KEY	Inngest request signing
NEXT_PUBLIC_APP_URL	Building webhook URLs for Google Form, Stripe, Telegram
UPSTASH_REDIS_URL	BullMQ cron queue and cron worker (ioredis format)
UPSTASH_REDIS_REST_URL	Webhook idempotency and rate limiting
UPSTASH_REDIS_REST_TOKEN	Webhook idempotency and rate limiting
RATE_LIMIT_GOOGLE_FORM_PER_MIN	Google Form webhook limit, default 30
RATE_LIMIT_STRIPE_PER_MIN	Stripe webhook limit, default 120
RATE_LIMIT_TELEGRAM_PER_MIN	Telegram webhook limit, default 60

If Redis is not configured
Without UPSTASH_REDIS_URL, cron schedules are silently not registered and the cron worker exits. Without the REST URL/token, webhook rate limiting and idempotency are disabled: duplicate deliveries can start duplicate runs.

8. Troubleshooting and Known Limitations

8.1 Symptom to cause

Symptom	Likely cause	Fix
Scheduled workflow never runs	Schedule not saved, Redis missing, or cron worker not running	Press workflow Save, set UPSTASH_REDIS_URL, run npm run cron-worker
Cron runs at the wrong time	Wrong timezone or wrong cron field order	Check the timezone dropdown and use minute hour day month weekday
Old schedule still fires after editing	Removing one schedule does not remove its scheduler key	Delete and re-add the Cron Trigger node, or clean the key manually
Telegram trigger does not fire	Webhook not registered, URL not public, or overwritten by another workflow	Save workflow, re-register webhook, check NEXT_PUBLIC_APP_URL
Register Webhook says credential missing	Node configuration was not saved to the database	Press workflow Save first, then Register Webhook
Telegram send fails with chat not found	Bot not started, not in group, or wrong chat id	Start the bot, add it to the chat, verify the id with getUpdates
Telegram can't parse entities	Unsupported HTML/Markdown in the message	Use Parse Mode None, or clean the AI output; the node retries plain text once
Google Sheets 403	API not enabled or sheet not shared with the service account	Enable Sheets API and share the sheet with client_email
Stripe variables are empty	Using top-level fields that the route does not create	Use {{stripe.raw.amount}}, {{stripe.raw.currency}}, or {{json stripe.raw}}
HTTP node fails on a 404/500	ky throws on non-2xx by default	Call an endpoint that returns 2xx, or extend the executor to handle errors
Execute workflow button is missing	No Manual Trigger node	Add a Manual Trigger, or use another trigger
Workflow contains a cycle	Connections form a loop	Remove the edge that closes the loop
Credential creation is forbidden	No active subscription	Upgrade; credential creation uses premiumProcedure

8.2 Known limitations and implementation notes

- Stripe webhooks are not signature-verified. The signing secret is not used by the route.
- Telegram callback buttons are acknowledged but do not start or branch a workflow.
- Removing a single schedule from an existing Cron Trigger node leaves its old BullMQ scheduler in Redis. Node deletion and workflow deletion clean up correctly.
- The HTTP Request node has no custom headers or authentication field.
- Discord and Slack dialogs suggest output paths that do not match the executor; the real key is messageContent.
- The OpenAI executor publishes its missing-credential error to the Anthropic realtime channel. This is a minor status-display bug.
- All nodes in a workflow execute, including disconnected nodes, because the topological sort adds self-edges for unconnected nodes.
- Only one Manual Trigger and one Cron Trigger are allowed per workflow by the editor. Other trigger types can be duplicated.
- The Telegram webhook route handles only message updates. Edits, reactions, and other update types are ignored.
- Webhook rate limiting and idempotency require Upstash REST credentials; without them the checks are skipped.

8.3 Recommended build order

1. Create the workflow and add the trigger.
2. Configure the trigger and press the workflow Save button.
3. For Telegram, reopen the trigger and Register Webhook after saving.

4. Add action nodes and connect them in order.
5. Configure each action node. Use the correct credential type.
6. Press Save. Confirm the Saved indicator.
7. For manual workflows, click Execute workflow.
8. For cron, start the cron worker and wait for the schedule.
9. For webhooks, send a test event from the external service.
10. Open Executions to inspect status, output, and errors.

Appendix A. Quick Reference

Node	Type	Key configuration	Output
Manual Trigger	MANUAL_TRIGGER	None	none
Schedule / Cron	CRON_TRIGGER	Schedules, timezone, dates	{{cron}}
Google Form	GOOGLE_FORM_TRIGGER	Webhook URL in Apps Script	{{googleForm}}
Stripe	STRIPE_TRIGGER	Webhook URL in Stripe	{{stripe}}
Telegram Trigger	TELEGRAM_TRIGGER	Bot credential + Register Webhook	{{telegram}}
HTTP Request	HTTP_REQUEST	Variable, method, URL, body	{{var.httpResponse}}
OpenAI	OPENAI	Variable, credential, prompts	{{var.text}}
Anthropic	ANTHROPIC	Variable, credential, prompts	{{var.text}}
Gemini	GEMINI	Variable, credential, prompts	{{var.text}}
Discord	DISCORD	Variable, webhook URL, content	{{var.messageContent}}
Slack	SLACK	Variable, webhook URL, content	{{var.messageContent}}
Send Telegram Message	TELEGRAM_SEND_MESSAGE	Variable, credential, chat id, text, parse mode	{{var.telegramMessage}}
Send Telegram Buttons	TELEGRAM_SEND_BUTTONS	Same + buttons (label, callback value)	{{var.telegramMessage}}
Google Sheets	GOOGLE_SHEETS	Variable, credential, operation, id, range, values	{{var.googleSheets}}

Appendix B. Source Files

Area	Files
Execution engine	src/inngest/functions.ts, src/inngest/utils.ts, src/features/executions/lib/executor-registry.ts
Node UI and dialogs	src/features/executions/components/*, src/features/trigger/components/*
Workflow persistence	src/features/workflows/server/routers.ts, src/features/editor/components/editor-header.tsx
Cron and BullMQ	src/lib/queue/cron.ts, src/lib/queue/connection.ts, src/queue/cron-worker.ts, src/features/trigger/components/cron-trigger/*
Webhooks	src/app/api/webhooks/google-form/route.ts, src/app/api/webhooks/stripe/route.ts, src/app/api/telegram/webhook/[workflowId]/route.ts
Credentials and encryption	src/features/credentials/*, src/lib/encryption.ts
External clients	src/lib/telegram.ts, src/lib/google-sheets.ts, src/lib/telegram-markdown.ts

End of guide. Generated from the Nodebase codebase. Behavior may change as the codebase evolves; re-check the source files listed above when in doubt.